



HANDWRITTEN_DIGIT_REC OGNITION



علاء صلاح مرشد عبده قائد

MAY 25, 2026

توثيق تطبيق التعرف على الأرقام المكتوبة بخط اليد :

□□ مقدمة عن المشروع

مشروع متكامل للتعرف على الأرقام المكتوبة بخط اليد باستخدام شبكة عصبية اصطناعية متعددة الطبقات (MLP) مبرمجة في PHP، مع دعم تدريب سريع وفَعَال عبر Python (NumPy) وتصدير النموذج إلى Laravel. يتيح التطبيق رسم الأرقام في المتصفح والتعرف عليها فوراً.

المميزات الرئيسية:

- واجهة تفاعلية متجاوبة (رسم مباشر للأرقام)
- شبكة عصبية بعمق حقيقي (طبقتان مخفيتان)
- تدريب سريع جداً عبر Python/NumPy (بدلاً من PHP البطيء)
- تصدير النموذج إلى JSON واستخدامه في Laravel للتنبؤ الفوري
- دعم جميع الأجهزة والمتصفحات
- شرح كامل للخوارزمية والهيكلية

نتائج عملية من التعرف على الأرقام :



Figure 1: Recognition number "2"

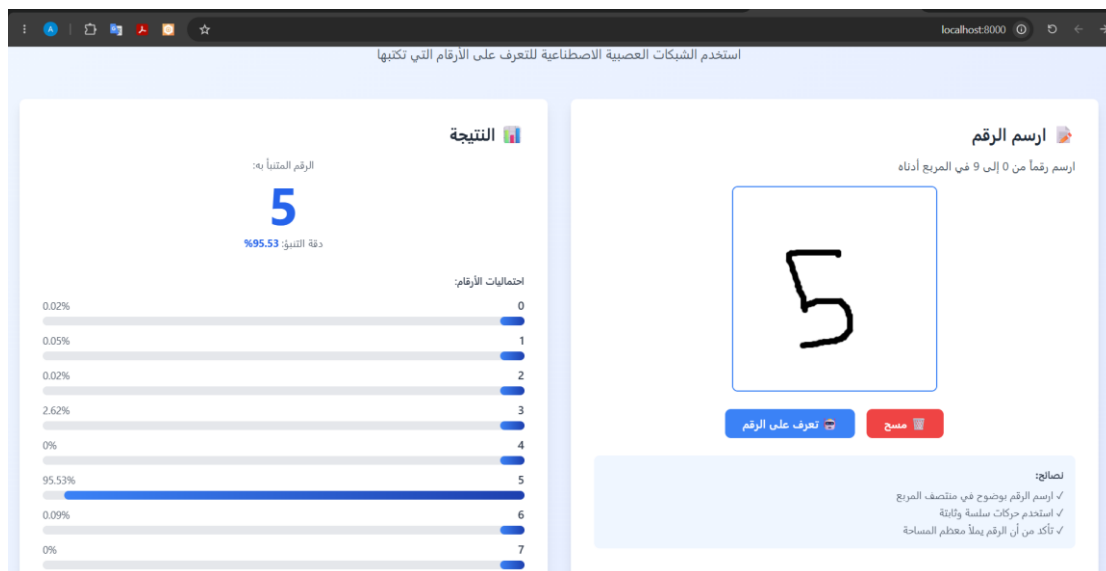


Figure 2: Recognition number "5"



Figure 3:Recognition number "7"



Figure 4:Recognition number "9"

هيكل المشروع 🏠

handwritten_digit_recognition/

- |— app/
 - | |— Http/
 - | | |— Controllers/
 - | | | |— DigitRecognitionController.php # المتحكم الرئيسي
 - | | |— Services/
 - | | | |— NeuralNetwork.php # فئة الشبكة العصبية
- |— routes/

- | |— web.php # المسارات
- |— resources/
 - | |— views/
 - | | |— digit-recognition/
 - | | | |— index.blade.php # الواجهة الأمامية
- |— storage/
 - | |— app/
 - | | |— neural_network_model.json # نموذج الشبكة المدرب
- |— train_mnist_python.py # ملف التدريب

عمل خطوات التثبيت والتشغيل والتدريب

المتطلبات:

- PHP 8.1 أو أحدث (للتشغيل)
- Composer
- Laravel 10 أو أحدث
- مكتبة PHP GD (لمعالجة الصور)
- Python 3.8+ مع مكتبة numpy (للتدريب السريع)

خطوات التثبيت والتشغيل:

1. استنساخ أو نسخ المشروع

```
# إذا كنت تستخدم Git
git clone <repository-url>
cd handwritten_digit_recognition
```

أو انسخ الملفات يدويًا إلى مشروعك #

2. تثبيت المكتبات

```
composer install
```

3. إنشاء ملف env.

```
cp .env.example .env
```

4. توليد مفتاح التطبيق

```
php artisan key:generate
```

5. تشغيل خادم التطوير

php artisan serve

ثم افتح: http://localhost:8000

تدريب النموذج (سريع واحترافي)

لماذا Python؟

تدريب الشبكة العصبية على بيانات كبيرة (مثل MNIST) في PHP بطيء جداً. لذلك يتم التدريب في Python (NumPy) ثم تصدير الأوزان إلى JSON ليتم استخدامها في Laravel/PHP للتنبؤ فقط.

خطوات التدريب:

1. تأكد من وجود Python وnumpy:

conda install numpy

أو

pip install numpy

2. نزل ملفات MNIST (train-images-idx3-ubyte.gz و train-labels-idx1-

ubyte.gz) وضعها بجانب train_mnist_python.py

- روابط رسمية: <https://storage.googleapis.com/cvdf-datasets/mnist/>

- أو من GitHub: <https://github.com/fgnt/mnist/tree/master>

3. شغل سكربت التدريب:

python train_mnist_python.py

4. بعد انتهاء التدريب سيظهر ملف `neural_network_model.json`

5. انسخ هذا الملف إلى `storage/app` في مشروع Laravel

6. الآن سيستخدم التطبيق النموذج المدرب فورياً للتنبؤ

ملاحظات حول الدقة:

- كلما زادت العصور (epochs) وحجم البيانات زادت الدقة (لكن الوقت يزداد)

- الشبكة الحالية (784 → 128 → 64 → 10) تحقق دقة عالية جداً على MNIST (>97%)

- لضمان أفضل دقة: درّب على كامل بيانات MNIST، epochs بين 10-30، batch=64 أو 128

□ بنية النموذج العصبي (Neural Network Architecture)

الشبكة المستخدمة هي MLP (Multi-Layer Perceptron) بترتيب:

- المدخلات: 784 (صورة 28x28)

- الطبقة المخفية الأولى: 128 وحدة (Sigmoid)

- الطبقة المخفية الثانية: 64 وحدة (Sigmoid)

- المخرجات: 10 (أرقام 0-9، Softmax)

كل الأوزان والانحيازات تحفظ في ملف JSON (متوافق بين Python و PHP).

1. فئة الشبكة العصبية `app/Services/NeuralNetwork.php` :

تحتوي على تطبيق كامل لشبكة MLP بعمق طبقتين مخفيتين:

- hiddenWeights (128→784)

- hidden2Weights (64→128)

- outputWeights (10→64)

مع دوال: الانتشار الأمامي، softmax، التنبؤ، تحميل/حفظ النموذج.

2. سكربت تدريب النموذج `train\_mnist\_python.py` :

سكربت Python يقوم بتدريب نفس بنية الشبكة العصبية على بيانات MNIST بسرعة عالية جداً، ثم يصدر النموذج إلى neural_network_model.json متوافق مع PHP.

3. المتحكم

`app/Http/Controllers/DigitRecognitionController.php`

يدير جميع طلبات التطبيق ومعالجة البيانات.

الدوال الرئيسية:

1. عرض الصفحة الرئيسية

```
public function index()
{
    return view('digit-recognition.index');
}
```

2. معالجة طلب التعرف

```
public function recognize(Request $request)
{
    // استقبال صورة Base64
    // تحويلها إلى مصفوفة بكسلات
    // التنبؤ برقم الصورة
    // إرجاع النتيجة كـ JSON
}
```

3. تحويل بيانات الصورة

```
private function imageDataToPixels(string $imageData): array
{
    // 1. فك تشفير Base64
    // 2. إنشاء صورة من البيانات
    // 3. تغيير الحجم إلى 28×28
    // 4. تحويل إلى تدرج الرمادي
    // 5. استخراج قيم البكسلات
    // 6. تطبيع القيم بين 0 و 1
}
```

4. تدريب النموذج

```
private function trainDefaultModel(): void
{
    // إنشاء بيانات تدريب افتراضية
    // تدريب الشبكة
    // حفظ النموذج
}
```

4. المسارات `routes/web.php`

تعريف جميع مسارات التطبيق:

// الصفحة الرئيسية

```
Route::get('/', [DigitRecognitionController::class, 'index'])->name('home');
```

// معالجة طلب التعرف على الرقم

```
Route::post('/api/recognize', [DigitRecognitionController::class, 'recognize'])->name('recognize');
```

// إعادة تدريب النموذج

```
Route::post('/api/retrain', [DigitRecognitionController::class, 'retrain'])->name('retrain');
```

// الحصول على معلومات النموذج

```
Route::get('/api/model-info', [DigitRecognitionController::class, 'modelInfo'])->name('model-info');
```

5. الواجهة الأمامية `resources/views/digit-recognition/index.blade.php` :

تطبيق الواجهة الأمامية باستخدام HTML و CSS و JavaScript.

المكونات الرئيسية:

1. Canvas للرسم

```
<canvas id="canvas" width="280" height="280"></canvas>
```

2. أزرار التحكم

- زر المسح: يمسخ Canvas
- زر التعرف: يرسل الصورة للخادم

3. عرض النتائج

- الرقم المتوقع به
- دقة التنبؤ (Confidence)
- احتماليات كل رقم

4. JavaScript للرسم والتفاعل

```
// تفعيل الرسم بالماوس
canvas.addEventListener('mousedown', ...);
canvas.addEventListener('mousemove', ...);
canvas.addEventListener('mouseup', ...);
// دعم اللمس للأجهزة المحمولة
canvas.addEventListener('touchstart', ...);
canvas.addEventListener('touchmove', ...);
canvas.addEventListener('touchend', ...);
// إرسال الصورة للخادم
recognizeBtn.addEventListener('click', async () => {
  const imageData = canvas.toDataURL('image/png');
  const response = await fetch('/api/recognize', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ image: imageData })
  });
});
```

□ شرح الخوارزمية (MLP)

1. معالجة الصورة

عندما يرسم المستخدم رقماً:

1. يتم تحويل صورة Canvas إلى Base64

2. إرسالها إلى الخادم

3. فك تشفير Base64 إلى بيانات ثنائية

4. تحويل إلى صورة باستخدام GD

5. تغيير الحجم إلى 28×28 بكسل

6. تحويل إلى تدرج الرمادي

7. استخراج قيم البكسلات (0-255)

8. تطبيع القيم بين 0 و 1

2. الانتشار الأمامي (Forward Propagation)

المدخلات (784) → طبقة مخفية أولى (128) → طبقة مخفية ثانية (64) → المخرجات

(10)

كل طبقة مخفية تستخدم دالة التنشيط (Sigmoid):

$$\text{hidden1} = \text{sigmoid}(W1 * \text{input} + b1)$$

$$\text{hidden2} = \text{sigmoid}(W2 * \text{hidden1} + b2)$$

$$\text{output_logits} = W3 * \text{hidden2} + b3$$

$$\text{probabilities} = \text{softmax}(\text{output_logits})$$

3. حساب الاحتماليات

$$\text{softmax}(x_i) = \exp(x_i) / \sum_j(\exp(x_j))$$

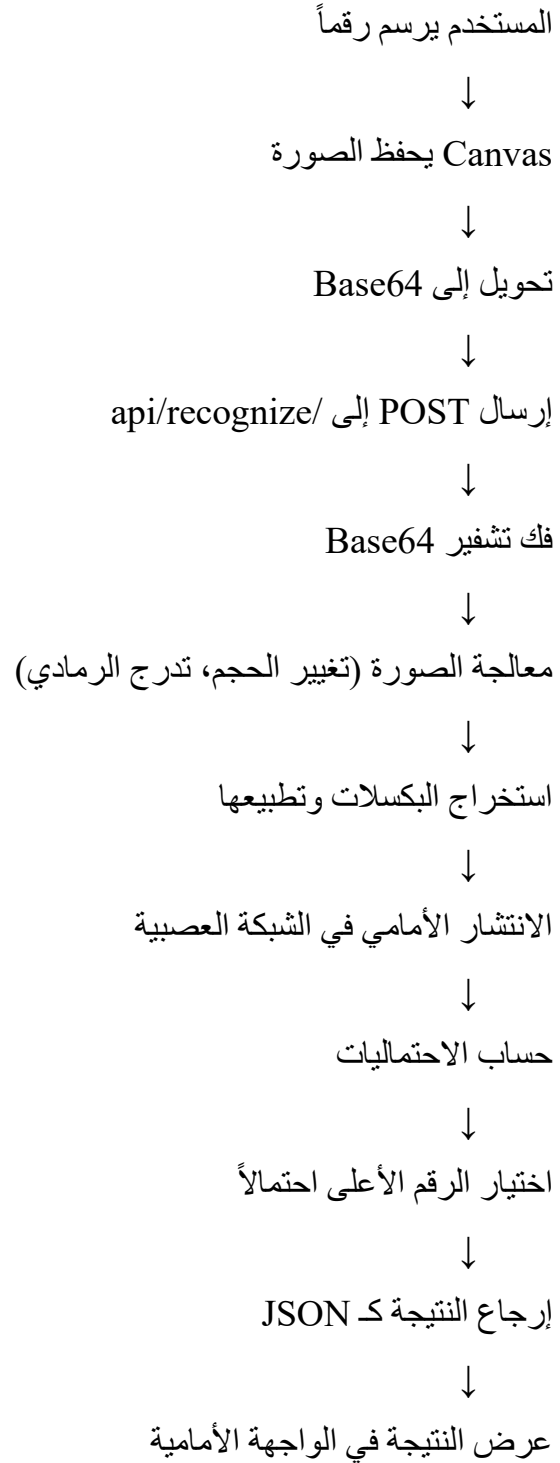
4. التنبؤ

$$\text{predicted_digit} = \text{argmax}(\text{probabilities})$$

معاملات الشبكة العصبية

المعامل	القيمة	الشرح
حجم المدخلات	784	صورة 28×28 بكسل
الطبقة المخفية 1	128	أول طبقة مخفية (sigmoid)
الطبقة المخفية 2	64	ثاني طبقة مخفية (sigmoid)
حجم المخرجات	10	أرقام من 0 إلى 9
معدل التعلم	0.1	سرعة تحديث الأوزان في Python
عدد الدورات	10-30	عدد مرات التدريب على البيانات
دالة التنشيط	Sigmoid	تحويل القيم إلى [0, 1]
دالة المخرجات	Softmax	تحويل القيم إلى احتمالات

🔄 تدفق البيانات



□ اختبار التطبيق

1. اختبار الرسم

- ارسم أرقاماً مختلفة في Canvas
- تأكد من أن الرسم يظهر بشكل صحيح

2. اختبار التعرف

- اضغط على زر "تعرف على الرقم"
- تحقق من أن النتيجة تظهر بشكل صحيح

3. اختبار الاستجابة

- جرب التطبيق على أجهزة مختلفة (هاتف، تابلت، حاسوب)
- تأكد من أن الواجهة متجاوبة

4. اختبار الأداء

- اختبر مع عدة صور متتالية
- تحقق من سرعة المعالجة

! تحسينات مستقبلية

1. استخدام مجموعة بيانات حقيقية: استخدام MNIST dataset للتدريب الفعلي
2. شبكات أعمق: إضافة طبقات مخفية أكثر
3. خوارزميات متقدمة: استخدام CNN (Convolutional Neural Networks)
4. حفظ النموذج في قاعدة البيانات: بدلاً من JSON
5. واجهة إدارية: لتدريب النموذج وتحديثه
6. API عام: للاستخدام من تطبيقات أخرى

🔧 أوامر التدريب والتشغيل

تدريب النموذج (سريع)

ملاحظة : هذا الامر سيعتمد مباشرةً على مكاتب مثبتته محليا في بيئة ال anaconda ويدرب النموذج ب 40 epochs .

```
python train_mnist_python.py
```

```
storage/app إلى neural_network_model.json ثم انسخ #
```

تشغيل التطبيق

```
php artisan serve
```

```
http://localhost:8000 ثم افتح #
```

🔧 التغييرات الأخيرة للتحسين من الأداء :

- تم زيادة عدد ال epochs في `train\_mnist\_python.py` إلى `40` وعاد التدريب مكتملاً بنجاح.

- نتيجة التدريب النهائية (Train accuracy) : 98.60%

- نموذج مُصدّر إلى `storage/app/neural\_network\_model.json` بعد الانتهاء.

- تم إجراء تقييم تجريبي للموديل على مجموعة الاختبار MNIST (عينة أو كاملة حسب توفر الملفات):

- مثال سابق لقياس دقة الاختبار أعطى `test\_acc = 93.98%` لنموذج سابق. (تختلف القيم حسب إعدادات التدريب)

- تحسينات المعالجة في الخادم (`DigitRecognitionController.php`) شملت:

- `findBoundingBox()` لتحديد مربع الرسم حول الرقم بدقة أعلى (عتبة افتراضية

220).

- `resizeToFit()` لتوسيط المحتوى داخل صندوق 20×20 قبل وضعه داخل 28×28.
- `imageToPixelArray()` لجمع بكسلات 28×28 وتطبيعها بحيث يكون الأسود قيمة عالية كما في MNIST.
- `centerPixelMass()` لتحريك الكتلة الداكنة إلى منتصف الصورة (محاذاة مركز الكتلة مثل MNIST).
- بعد هذه التعديلات، تجارب يدوية أظهرت تحسناً واضحاً في التعرف داخل الواجهة — الأخطاء النادر أصبحت أقل بكثير.

✓ أوامر صيانة وتحديث نفذت أثناء العمل

بين الأوامر التي نُفذت لتفريغ التخزين المؤقت وضمان تحميل التغييرات حديثاً في Laravel:

```
php artisan cache:clear
php artisan route:clear
php artisan view:clear
php artisan config:clear
php artisan optimize:clear
php artisan auth:clear-resets
php artisan event:clear
php artisan queue:clear
php artisan schedule:clear-cache
```

```
php artisan clear-compiled
php artisan session:table
```

```
composer dump-autoload
```

استخدم هذه الأوامر بعد استبدال `neural\_network\_model.json` أو عند تعديل ملفات PHP/Blade لضمان أن التغييرات سارية.

🔄 كيف تعيد تدريب النموذج وتثبتته بسرعة

1. ضع ملفات MNIST (`train-\*.gz` و `t10k-\*.gz`) بجانب
`train\_mnist\_python.py`.
2. شغّل التدريب (مثال مع 40 epochs):

```
python train_mnist_python.py
```